

# Narramancer Getting Started

## Node graphs with save and load built in

Version 1.0 · Unity 6 · Full documentation at **narramancer.com**

# Contents


1 What Narramancer is

2 Installing

3 The samples

4 The building blocks

5 The Narramancer window

6 Saving and loading — the part to read twice

7 Upgrading from 0.5


## 1. What Narramancer is

Narramancer lets you build a game's story and logic as node graphs. Its main feature is that a save can be made **at any moment**: a graph halfway through an action is saved, loaded back, and carries on from exactly where it stopped.

Use it for branching dialogue, quest and story flow, NPC behaviour, or the overall loop of your game. Each graph is a chain of small steps: show this text, wait for a click, offer these choices, move this character, wait two seconds. Some steps finish at once and some take time. Narramancer can save a graph while it is waiting on any of them.

Most scripting tools can't do that. Unity Visual Scripting and Fungus have no way to save which step is running, so saving mid-sequence is something you would build yourself. In Narramancer it is how graphs run.

### Why use it

- **Save anywhere, from day one.** Saving is not bolted on later. Every graph is saveable from the start.
- **Logic you can see.** The flow of a scene is laid out in a graph, not spread across scripts.
- **A built-in database of things.** People, places and items, with their properties, stats and relationships, all saved for you.
- **Little or no code.** Over 150 node types cover text, choices, variables, lists and scene objects.
- **Iterate in Play Mode.** Edit graphs and test ideas without recompiling.

# 2. Installing

1. In Unity, open **Window → Package Manager**, choose **My Assets**, find Narramancer and click **Import**.
2. Open **Window → Narramancer** once. This creates the **Narramancer** settings asset in `Assets/Resources`, which every other part of the system reads from.

That is all the setup there is. Narramancer has no other package dependencies and needs no Player Settings changes.

| Requirement | Details |
|---|---|
| Unity version | Unity 6. Built and tested on 6000.4. |
| UI | The sample printers and menus use Unity UI (uGUI) with legacy `Text`. |
| Navigation (optional) | The NavMesh features, such as the **Move NavMesh Agent** node, need the built-in **AI** module. It is on by default; if you turn it off, those features are left out of the build and everything else keeps working. |
| Odin Inspector | Not required, and it no longer conflicts. Narramancer has its own serializer and ships no copy of Odin. |

> **Upgrading from 0.5?** The ".NET 4.x API compatibility" step in older guides is no longer needed. Chapter 7 covers what else changed.

# 3. The samples

The samples are in `Narramancer/Scenes`, one folder each. Open any scene and press Play.

| Sample | What it shows |
|---|---|
| **0. Sample Launcher** | A menu that opens the other four, so a single build can show all of them. Each sample gets a ◀ **Samples** button to come back. |
| **1. Text and Choices Example** | Text revealed a line at a time, waiting for a click, then a choice that branches the story. Start here. |
| **2. Noun Creation and Destruction** | Creating and destroying nouns while the game runs. |
| **3. Adding and Removing Properties** | Changing a noun's properties at runtime and reacting to them. |
| **4. Character Equipment** | Equipment slots built from relationships and stats, with UI that verbs update: sprites, titles and descriptions, all saved. |

Every sample has **Save** and **Load** buttons in the corner. Try saving halfway through a conversation, then loading: the text, the choices and the graph all pick up where they were.

### Running the samples in a build

Build Settings belong to your project, not to the package, so add the scenes yourself. Open **File → Build Profiles** and add **0. Sample Launcher** first, then samples 1 to 4. The launcher lists whatever else is in the build, so a sample shows up in it by being added there.

### How sample 1 is put together

Select the **Narramancer Scene** object in the hierarchy. Its **Run on Start Verbs** list holds one ActionVerb, *Text and Choices Example Verb*, and that one graph is the whole sample. Double-click it to open it in the graph editor. The text box and the choice buttons come from the **Text and Choice Printer Canvas** prefab, and the save and load buttons from **Overlay Buttons Manager Canvas**. Copy both prefabs into your own scene to get the same UI.

# 4. The building blocks

Narramancer names its parts after grammar. Once that clicks, the rest of the system follows.

| Part | What it is |
|---|---|
| **Nouns** | The things in your game: characters, items, places, ideas. A noun asset is the template; at runtime each one becomes a *noun instance* with its own values. |
| **Verbs** | What things *do*, authored as node graphs. An **ActionVerb** runs over time and can wait (for a click, a choice, a timer). A **ValueVerb** answers a question instantly and returns a value. ActionVerbs can use ValueVerbs, but not the other way round. |
| **Adjectives** | Data attached to nouns: **Properties** are tags (*Poisoned*, *Armor*), **Stats** are numbers (*Health*, *Damage*), and **Relationships** are one-way links from one noun to another (*equipped by*, *friend of*). |
| **Variables** | Named, typed values that graphs read and write. |

## Variables

A variable has a name, a type and a **scope**, which sets where it lives:

- **Global:** one value for the whole game, defined in the Narramancer window.
- **Scene:** defined on a **Narramancer Scene** component and available while that scene is loaded.
- **Verb:** a verb's own inputs and outputs.

Read a variable with a **Get Variable** node and write one with a **Set Variable** node. Set the node's scope to match the variable. Set Variable only works inside an ActionVerb, and only changes the value when it runs.

Global variables can have a starting value in the Narramancer window, but that window can't point at objects in a scene. To give a verb a scene object, use one of these three:

1. A **Set Global Variables** component, which assigns global variables when the scene starts.
2. A **Narramancer Scene** component, with the object assigned to one of its scene variables.
3. A **Run Action Verb** component, with the object assigned to one of the verb's inputs.

All three are under **GameObject → Narramancer**. After changing a verb's inputs or the global variables, press **Update Assignments** on a Run Action Verb or Set Global Variables component to sync it. Values assigned to a renamed input or variable are kept, as long as its type is unchanged.

# 5. The Narramancer window

Open it from **Window → Narramancer**. It docks like any other panel and is the hub for your game's starting nouns, verbs and global variables. (It is really the inspector of the **Narramancer** settings asset in `Assets/Resources`.)

### Editor mode

Outside Play Mode the window has four tabs: **Nouns**, **Adjectives**, **Verbs** and **Variables**. Nouns listed here are created when the game starts, and verbs listed here run right after. The Variables tab is where global variables are defined and given starting values.

### Runtime mode

In Play Mode the window shows the live table of noun instances. You can inspect any of them and add or remove properties and stats while the game runs, which is handy for testing a branch without playing up to it.

### From code

Everything the window shows is also available from scripts through one entry point:

```
var narramancer = NarramancerSingleton.Instance;   // exists once the game starts
```

# 6. Saving and loading

Most of a save happens on its own. The one thing you decide is which scene objects are saved, and that is set on one component.

**The story**

Saved automatically

- Every noun instance, with its properties, stats and relationships
- Every variable
- Every running verb, including exactly what it is waiting on: a click, a choice, a timer
- The text and choices on screen

**Scene objects**

Saved where you add a Saveable

- Position, rotation and scale
- Images, sprites, text and audio
- Whether the object is active
- Fields of your own scripts marked `[Save]`

## Adding save and load to your game

The quickest way is the **Overlay Buttons Manager Canvas** prefab from the samples: Save and Load buttons, save slots with thumbnails, and a history log. From your own code, a save and a load look like this:

```
// Save
var story = NarramancerSingleton.Instance.PrepareStoryForSave();
SaveLoadUtilities.WriteSaveData("slot1", SaveLoadUtilities.SerializeData(story));

// Load: reopens the scene the save was made in, then restores the story into it
var save = SaveLoadUtilities.ReadSaveWrapper("slot1");
NarramancerSingleton.Instance.LoadStory(save);
```

Saves are JSON files. In the editor they go to `Assets/Saves` so you can open and read them; in a build they go to `Application.persistentDataPath/Saves`. On WebGL they are stored in the browser and survive a page reload.

## Saving scene objects: the Saveable component

Add one **Saveable** to each GameObject whose state should come back after a load. It does two things:

- **It gives the object a permanent id.** Saves point at objects by this id, not by name or place in the hierarchy. You can rename, reparent or reorder an object without breaking existing saves.
- **It saves what you tick.** The inspector lists **Active state**, plus a toggle for each component on the object that Narramancer knows how to save:

| Component | What comes back after a load |
|---|---|
| Transform | Local position, rotation and scale |
| Rect Transform | Anchored position, size, rotation and scale |
| Image, Sprite Renderer | Sprite, color and whether it is enabled |
| Text | The text, its color and whether it is enabled |
| Audio Source | Clip, playback position, volume, pitch, looping and whether it was playing |
| Animation | Clip, time, speed and wrap mode |
| Nav Mesh Agent | Destination, speed, stopping distance and whether it is stopped |

Adding a Saveable by hand ticks everything it finds. Untick what should reset to the scene's own state on a load. A Saveable with nothing ticked is **identity only**: other saves can point at the object, but nothing about the object itself is saved.

To add Saveables in bulk, run **Window → Narramancer → Add Missing Saveables** (open scenes, or prefabs). It finds objects that need an id because something points at them, and adds identity-only Saveables. Tick what you want saved afterwards.

> **If a verb changes it, save it.** A verb can set a label's text, swap an image or hide an object. If that object's Saveable doesn't save that component, or the object has no Saveable, the change is undone by the next load: the scene comes back as it was built, not as the verb left it. When a label or image looks wrong after a load, check this first.

## Your own scripts

Mark a field `[Save]` on any MonoBehaviour that sits on an object with a Saveable, and it is saved and restored with the object. No other code is needed. The inspector lists these scripts under **Also saved, by their [Save] fields**.

```
public class Lantern : MonoBehaviour {
    [Save] private float oilLeft = 1f;
    [Save] private bool lit;
    [Save] private List<string> litBy = new List<string>();
}
```

Fields can hold numbers, strings, Unity value types like `Vector3` and `Color`, lists, dictionaries, your own `[Serializable]` classes, assets, and scene objects that have a Saveable.

## Movement and animation

The **Translate GameObject**, **Tween GameObject Rotation**, **Play Animation** and **Move NavMesh Agent** nodes keep going after a load: a character saved halfway through a walk finishes the walk, and a verb waiting for it to arrive carries on when it does. The moving object needs a Saveable. If it doesn't have one, Narramancer logs a warning when the move starts.

## Assets in a save

When a save points at an asset, such as the sprite on an image or a noun in a variable, it records a permanent id. In a build, those ids are looked up in an **asset manifest** that Narramancer rebuilds before every build. The manifest includes:

- every Narramancer asset (nouns, adjectives, verbs and their nodes),
- every asset those reference, such as the sprites in your nouns,
- everything in a `Resources` folder.

That covers almost everything. The exception is an asset that only your own code or the scene refers to. Add those to the manifest's **Additional Assets** list (`Assets/Resources/NarramancerAssetManifest`). If a save meets an asset it can't name, it logs a warning saying which one, and that one item comes back as the scene has it.

> **Test saves in a build before you ship.** Play Mode in the editor can look up any asset, so a save that works there can still miss something in a player. A WebGL or IL2CPP build is the real test. Save, quit, relaunch and load.

# 7. Upgrading from 0.5

Version 1.0 changes how saving works. Because of that, a project from 0.5 needs a few one-time fixes.

- **Odin Serializer is gone.** Delete any leftover `OdinSerializer` folder. The ".NET 4.x" Player Setting is no longer required.
- **Old save files can't be loaded.** The save format changed, and a 0.5 save is rejected with a message, not misread.
- **The `Serialize…` components were replaced by Saveable.** Scenes that used them will show *missing script* warnings where they were. Remove those, add a Saveable, and tick the matching toggle:

| 0.5 component | 1.0 equivalent |
|---|---|
| `SerializeTransform` | Saveable → **Transform** |
| `SerializeRectTransform` | Saveable → **Rect Transform** |
| `SerializeActive` | Saveable → **Active state** |
| `SerializeImage` | Saveable → **Image** |
| `SerializeSpriteRenderer` | Saveable → **Sprite Renderer** |
| `SerializeText` | Saveable → **Text** |
| `SerializeAudioSource` | Saveable → **Audio Source** |
| `SerializeAnimation` | Saveable → **Animation** |
| `[SerializeMonoBehaviourField]` | `[Save]` |
| `SerializeNounInstanceReference` | Unchanged |

Then run **Window → Narramancer → Add Missing Saveables** on your scenes and prefabs, so every object something points at has a permanent id.

Two more changes come for free. You no longer need to add a component to an object before moving or animating it from a node, because the nodes set that up themselves. And objects that start inactive are now saved like any other.

## Where to go next

This guide covers the essentials. The full documentation, including every node, is at **narramancer.com**. The *Narramancer Online Documentation* link in this folder opens it.